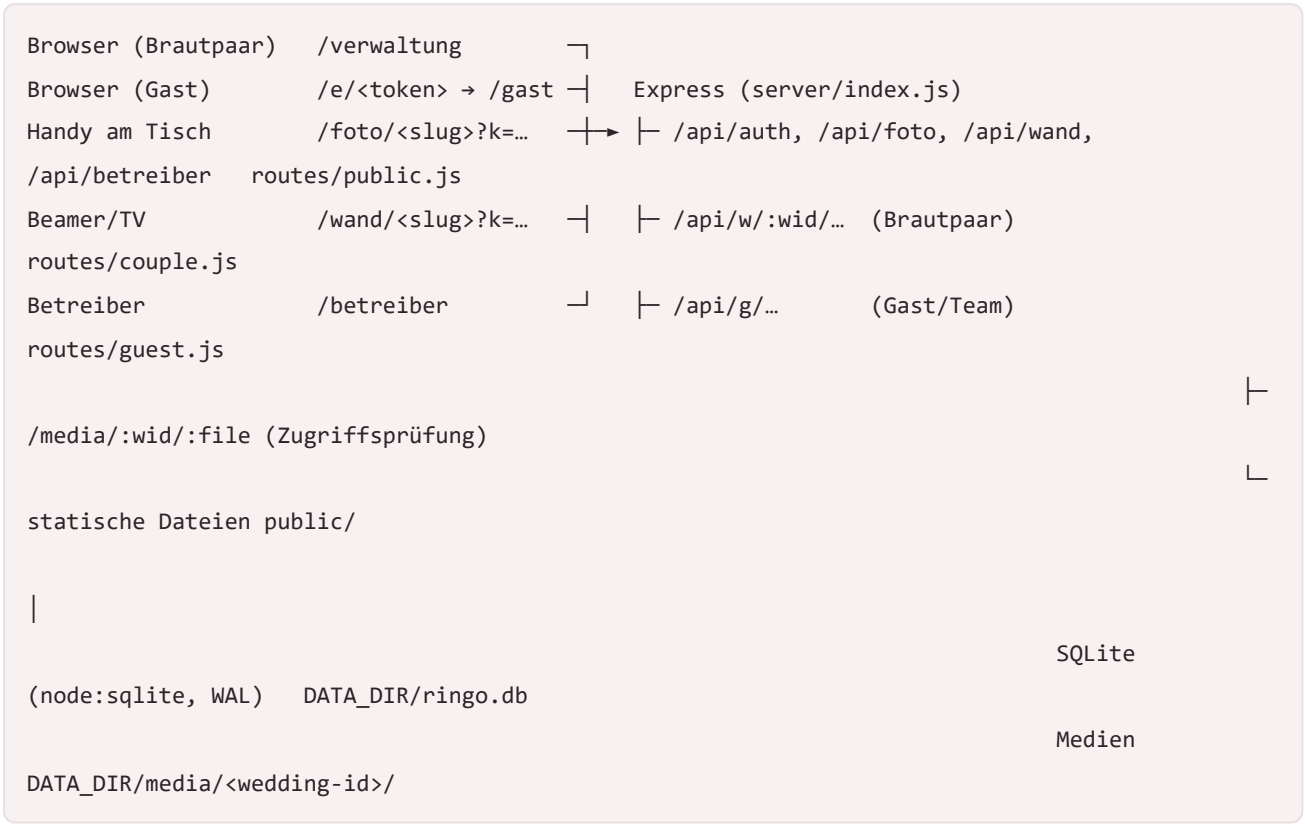
Architektur

Überblick



Bewusste Entscheidungen

- **Kein Build-Schritt, kein Framework im Frontend.** Native ES-Module, ein kleiner DOM-Helfer `h()` in `public/js/common.js`. Das Projekt bleibt so langlebig und leicht zu warten.
- **Rechenarbeit im Browser des Gastes.**
  - Fotos werden vor dem Upload auf 2400 px verkleinert, mit Vorschaubild.
  - Der Browser berechnet einen Schärfewert (Varianz des Laplace-Operators) und einen 64-Bit-Ähnlichkeits-Hash.
  - Optional erkennt er Gesichter (face-api.js, lokal ausgeliefert).
  - Der Server bleibt dadurch klein und braucht keine Bildbibliotheken.
- **Server-Sent Events** statt WebSockets für Live-Ablauf, neue Fotos und Nachrichten (`server/realtime.js`). Das funktioniert durch jeden Proxy, der Pufferung abschalten kann.
- **Keine externen Dienste nötig.** SMTP und Claude API sind optional, jede Funktion hat einen Rückfallweg ohne sie.

Datenmodell (Auszug)

Tabelle	Inhalt
`users`, `sessions`	Konten der Brautpaare
`weddings`	Eine Hochzeit. Enthält `slug`, `plan`, `upload_key` (Tisch-QR) und `wall_key` (Fotowand), außerdem Einstellungen für Fotos, Geschenke und den Text der Dankeskarten. `live_item_id` ist die manuelle Live-Markierung
`events`	Termine, `is_main` markiert den Haupttermin
`guests`	Gäste mit `token` (persönlicher Link), Rolle (`guest` / `witness` / `organizer`), RSVP-Details, `table_id`, optional `face_descriptor`
`invitations`	Gast × Termin mit `rsvp` (`open` / `yes` / `no`) und Begleitung
`program_items`	Ablauf. `status` ist `requested`, `confirmed` oder `declined`. `surprise` blendet den Punkt für das Brautpaar aus
`photos`, `photo_likes`	Fotos und Videos mit `approved`, `favorite`, `quality`, `ahash` und `faces` (JSON mit 128-d-Vektoren)
`guestbook`	Text-, Audio- und Videobotschaften
`gift_items`, `pledges`	Hochzeitskasse (`fund`) und konkrete Geschenke (`item`, mit `quantity`), dazu die Zusagen
`tables`, `seat_rules`	Sitzplan, Regeln `together` / `apart`
`faqs`, `questions`, `announcements`	Infos und Kommunikation
`team_posts`, `anecdotes`	Geheimer Team-Bereich, Geschichten für Reden

Design-Templates

Das Brautpaar wählt eines von acht Designs (`weddings.theme`, Werte in `server/logic.js` → `THEMES`). - **Geltung:** Das Design gilt für die Gäste-App, die Foto-Upload-Seite und die Fotowand. Die Verwaltung bleibt im Standarddesign. - **Anwendung im Browser:** `public/js/themes.js` setzt `<html data-theme="…">` und baut den passenden Kopfbereich, je Design ein eigener (Zeitungskopf, Sofortbilder, Aufgebot …). - **Gestaltung:** `public/css/themes.css` überschreibt nur Design-Variablen (Farben, Schriften, Radien, Rahmen) und einzelne Bausteine. - **„Klassisch“:** Es braucht keine eigenen Regeln. Alle Variablen in `app.css` haben die ursprünglichen Werte als Standard, das Design ist nach dem Umbau pixelgenau unverändert. - **Wechsel:** Er wird per Server-Sent Event (`theme`) an offene Gästeseiten und die Fotowand verteilt. - **Schriften:** Alle Schriften der Designs liegen selbst gehostet unter `public/fonts/themes/`. Der Browser lädt nur die Schriften des aktiven Designs. - **Eigenes Logo:** `weddings.logo_path`. Der Browser füllt das Bild auf ein transparentes Quadrat auf (PNG, max. 1024 px), SVG wird abgelehnt. `lettering()` in `themes.js` setzt es in jedem Design an die Stelle des Namensschriftzugs, `namesOrLogo()` in `server/pdf.js` auf den

Drucksachen. Der Tisch-QR-Schlüssel gibt neben dem Titelfoto auch das Logo frei. - **Vorschaubilder:** Die Bilder für die Auswahl (`public/img/themes/*.jpg`) erzeugt `node tests/theme-screens.mjs <ordner> --thumbs` .

Rechte- und Sichtbarkeitsmodell

Daten	Brautpaar	Team (Trauzeugen/Orga)	Gast	Tisch-QR (`upload_key`)	Fotowand (`wall_key`)
Termine	alle	nur eingeladene	nur eingeladene	–	–
Programm	ohne Überraschungen	alles	bestätigt, ohne Überraschungen, plus eigene Anfragen	–	Live-Status
Fotos	alle, Freigabe	freigegebene + eigene	freigegebene + eigene	nur hochladen	freigegebene
Gästebuch	alles	nur eigene	nur eigene	–	–
Geschenk-Zusagen	mit Namen (außer „anonym“)	Summen	Summen + eigene	–	–
Team-Bereich, Anekdoten	**nein**	ja	Anekdote einreichen	–	–

Der Live-Status wird **pro Gast** berechnet. Ein Gast sieht nur Programmpunkte der Termine, zu denen er eingeladen ist. Die Server-Sent Events sind ein Signal an alle Gäste der Hochzeit. Daraufhin lädt jeder Gast seinen eigenen, gefilterten Live-Status nach.

Wichtige Abläufe

Persönlicher Link: `/e/<token>` setzt das Cookie `hb_guest` (400 Tage) und leitet nach `/gast` weiter. Alle Gäste-APIs lesen den Gast aus diesem Cookie.

Foto-Upload: 1. Der Browser verkleinert das Bild und berechnet Schärfe und Hash. Bei aktivierter Suche kommen Gesichtsvektoren dazu. 2. Er schickt `multipart/form-data` mit `file` , `thumb` und den Metadaten an den Server. 3. Der Server prüft Typ, Größe und Paketgrenzen und speichert die Datei. 4. Ist das Foto freigegeben, sendet der Server das Ereignis `photo` an alle Clients.

Finde meine Fotos: 1. Der Gast macht ein Selfie. Der Browser berechnet daraus einen Vektor, das Selfie selbst bleibt auf dem Gerät. 2. Der Browser schickt den Vektor an `POST /api/g/face-search` . 3. Der Server sucht die Fotos mit einer euklidischen Distanz unter 0,5. 4. Nur wenn der Gast „merken“ wählt, wird sein Vektor gespeichert. Er dient dann für die Dankeskarten.

Sitzplan-Assistent (`server/seating.js`): 1. „Zusammen“-Regeln werden per Union-Find zu Clustern zusammengefasst. 2. Die Cluster werden nach Größe sortiert verteilt. Jeder kommt an den Tisch, der passt und keine „Getrennt“-Regel verletzt, mit Bonus für gleiche Gruppen. 3.

Begleitungen und Kinder belegen eigene Plätze.

Fotobuch-Auswahl: Zuerst die Favoriten, dann die Bilder nach Schärfe. Fast-Duplikate (Hamming-Distanz des Hashs ≤ 6) werden übersprungen.

KI (`server/ai.js`, optional): - Gästefragen: Antworten dürfen nur aus den Infos des Brautpaars kommen. Weiß das Modell es nicht, wird die Frage an das Paar weitergeleitet. - Rede-Helfer: Entwurf aus den gesammelten Anekdoten. - Modell `claude-opus-5-5` mit `effort` `low` bzw. `medium`. - Die serverseitigen Fallbacks der Claude API sind aktiviert.

API (Auszug)

Methode & Pfad	Zweck
`POST /api/auth/signup` · `login` · `logout` · `GET me`	Konto des Brautpaars
`GET/PATCH/DELETE /api/w/:wid`	Übersicht, Einstellungen, Hochzeit löschen
`/api/w/:wid/events`, `/guests`, `/guests/import`, `/guests/export.csv`, `/guests/send-invites`	Termine & Gäste
`/api/w/:wid/program`, `POST /live`	Ablauf & Live-Steuerung
`/api/w/:wid/photos`, `/archiv.zip`, `/fotobuch.pdf`	Fotos
`/api/w/:wid/gifts`, `/pledges/:id` · `/tables`, `/seat-rules`, `POST /seating/auto`	Geschenke, Sitzplan
`/api/w/:wid/faqs`, `/questions`, `/announcements`	Kommunikation
`/api/w/:wid/tischkarten.pdf`, `/aushang.pdf`, `/gaesteliste.pdf`, `/dankeskarten.pdf`	Drucksachen
`GET /api/g/me`, `POST /rsvp`, `/program`, `/photos`, `/guestbook`, `/gifts/:id/pledge`, `/seat`, `/ask`, `/face-search`, `/anecdotes`	Gäste-App
`GET /api/g/team`, `POST /team/posts`, `/team/speech`, `PATCH /program/:id`, `POST /live`	Team-Bereich
`GET /api/foto/:slug?k=`, `POST /api/foto/:slug/upload?k=`	Tisch-QR-Upload
`GET /api/wand/:slug?k=`, `/stream`	Fotowand

Schreibende Aufrufe brauchen den Header `X-Requested-With: ringo`.

Tests

- `tests/api.test.js` (26 Tests): Konten, CSRF, fremde Konten, CSV-Import, Einladungslink, RSVP, Programm-Rechte & Überraschungen, Live-Filter pro Gast, Upload & Medienzugriff (inklusive Path Traversal), Moderation, Fotowand-Schlüssel, Gesichtssuche, Geschenke & GiroCode-Format, Sitzplan, FAQ-Rückfall, Team-Bereich, Paketgrenzen, Dateitypen, Exporte, Betreiber-Konsole, vollständiges Löschen.

- `tests/e2e-screens.mjs`: Ruft jede Seite als Brautpaar, Gast, Team, QR-Uploader und Fotowand in Chrome auf und schlägt bei JavaScript-Fehlern oder HTTP-Fehlern fehl. Dazu laufen alle PDF-, CSV- und ZIP-Exporte.
- `tests/e2e-interactive.mjs`: Prüft die echte Bildverkleinerung samt Upload, die lokale Gesichtserkennung unter der CSP und die Video- und Sprachaufnahme mit simulierter Kamera.